GIBA Layered Build Plan with Functional Requirements and Required Functions

1. Purpose and Scope

This document defines a production build plan for the GIBA AI maintenance assistant with:

1. Functional requirements per system layer.
2. Required functions and interfaces to implement.
3. Example event flows for core and edge scenarios.

The plan is aligned with the current repository stack:

- Frontend: React, TypeScript, Vite, i18next.
- Backend: FastAPI, Pydantic settings, structlog, SQLAlchemy, Alembic.
- AI and retrieval: LangChain ecosystem, sentence-transformers, Chroma.
- Data and infrastructure: PostgreSQL, Redis, Docker Compose.

2. Current State Snapshot

Observed implementation status in this repository:

1. Backend bootstrapped with health endpoint, CORS, and request logging middleware.
2. AI service and AI client are present as stubs and must be implemented.
3. Frontend includes chat and protected route UX but is currently HR-themed and mock-driven.
4. Core routers for auth, chat, reports, and admin are placeholders in backend startup.

Implication:
The build plan below includes both completion of missing core functions and hardening requirements for production readiness.

3. Layered Architecture Plan

Layer A. Experience and Presentation Layer

Mission:
Provide one unified interface for repairers and admins to authenticate, submit structured reports, query solutions, and view citations.

Technology stack:

- React 19 with TypeScript.
- React Router for navigation and role-based route protection.
- i18next for multilingual support.
- Lucide icons and custom components for chat and form experiences.

Functional requirements:

1. Structured report workflow embedded in chat.
2. Required fields for report submission: problem, cause, solution, machine_type.
3. Review loop with Approve and Modify actions before persistence.
4. Chat response rendering with citation cards and confidence indicator.
5. Machine selector restricted to user allowed machines only.
6. Clear handling for unauthorized attempts and low-confidence AI responses.

Required frontend functions to implement:

1. authApi.login(credentials)
2. authApi.refreshToken()
3. reportApi.reformulateReport(payload)
4. reportApi.modifyReformulation(payload)
5. reportApi.commitReport(payload)
6. chatApi.querySolution(payload)
7. ingestionApi.submitManual(payload)
8. ingestionApi.submitManufacturerAlert(payload)
9. useAuthGuard(requiredRole?)
10. useMachineScope() to fetch and cache allowed machines
11. renderCitations(citations)
12. renderConfidenceBadge(score, level)

Suggested frontend components:

1. ReportComposerPanel
2. ReformulationReviewCard
3. CitationDrawer
4. ConfidenceBanner
5. UnauthorizedAccessNotice
6. IngestionJobStatusTable

Acceptance criteria:

1. User can complete reformulation loop without leaving chat.
2. Chat answer always shows source references when factual recommendations are present.
3. Unauthorized machine selection is blocked in UI and server-validated.

Layer B. API and Application Orchestration Layer

Mission:
Expose secure and versioned APIs that orchestrate authentication, report lifecycle, retrieval, AI generation, and ingestion jobs.

Technology stack:

- FastAPI and Uvicorn.
- Pydantic models for strict request and response contracts.
- Structlog for structured logs.
- Httpx for external APIs.

Functional requirements:

1. Token authentication and permission-aware request context.
2. Versioned endpoints for report reformulation, report commit, chat query, manufacturer alert webhook, and manual ingestion.
3. Idempotent ingestion endpoints.
4. Correlation IDs and structured logs across request lifecycle.
5. Central exception mapping to consistent API error format.

Required backend API routes:

1. POST /auth/login
2. POST /auth/refresh
3. GET /auth/me
4. POST /reports/reformulate
5. POST /reports/modify
6. POST /reports/commit
7. POST /chat/query
8. POST /ingestion/manufacturer-alert
9. POST /ingestion/manual
10. GET /ingestion/jobs/{job_id}
11. GET /health
12. GET /metrics

Required orchestration functions:

1. build_request_context(request)
2. validate_machine_access(user, machine_type)
3. enforce_scope_filters(user)
4. attach_correlation_id(request, response)
5. map_domain_error_to_http(error)
6. verify_idempotency_key(key, route)
7. store_idempotency_result(key, route, result)

Acceptance criteria:

1. All protected routes reject invalid or expired tokens.
2. Ingestion webhooks are replay-safe using idempotency keys.
3. Every request log includes path, status, duration, and correlation ID.

Layer C. AI, Prompting, and RAG Layer

Mission:
Generate grounded and safe maintenance guidance by combining user query intent with authorized context from reports, manuals, and manufacturer alerts.

Technology stack:

- LangChain orchestration.
- Provider client integration via AI client abstraction.
- Sentence-transformers for embedding generation.

Functional requirements:

1. Structured report reformulation preserving original meaning.
2. Natural-language modification loop over cleaned report fields.
3. Retrieval-augmented chat response with top-k relevant context.
4. Citation mapping between generated statements and retrieved chunks.
5. Confidence scoring and fallback clarifying-question mode.
6. Safety guardrails for high-risk actions.

Required AI service functions:

1. reformulate_report(problem, cause, solution)
2. modify_reformulation(clean_fields, user_instruction)
3. generate_query_embedding(query)
4. retrieve_context(query_embedding, scope_filter, top_k)
5. compute_confidence(retrieval_scores)
6. generate_grounded_answer(query, retrieved_context)
7. map_citations(answer, retrieved_chunks)
8. apply_safety_guardrails(answer, risk_rules)
9. build_clarification_prompt(query, missing_info)

Required class-level implementation in current stubs:

1. AIClient.**init** with provider setup and credentials.
2. AIClient.generate(messages, model, temperature, max_tokens).
3. GenrativeAIServices.generate_text(prompt) with actual provider call.
4. GenrativeAIServices methods for reformulation, modification, and grounded response.

Acceptance criteria:

1. Generated answers contain references to retrieved sources.
2. Low-confidence queries trigger clarification prompts instead of speculative advice.
3. Reformulation does not alter technical meaning of user input.

Layer D. Data and Knowledge Storage Layer

Mission:
Persist transactional business data and searchable semantic knowledge with strict metadata for secure filtering.

Technology stack:

- PostgreSQL with SQLAlchemy and Alembic.
- Chroma as vector store with metadata filtering.

Functional requirements:

1. Canonical report schema with raw and cleaned structured fields.
2. Knowledge source taxonomy: repairer, manufacturer, manual.
3. Mandatory metadata on each chunk: machine_type, source, timestamp.
4. Optional metadata for isolation and quality: plant_id, line_id, validation_status, section, page.
5. Audit records linking query, retrieved chunk IDs, and response ID.

Required persistence functions:

1. save_report(report_model)
2. save_clean_report_fields(report_id, clean_fields)
3. save_embedding_record(chunk_text, embedding, metadata)
4. query_embeddings(query_vector, metadata_filter, top_k)
5. save_audit_trace(trace_record)
6. save_ingestion_job(job_record)
7. update_ingestion_job_status(job_id, status, error?)
8. get_ingestion_job(job_id)

Acceptance criteria:

1. Metadata filtering guarantees no cross-scope retrieval leakage.
2. Migration scripts create and evolve schema deterministically.
3. Audit and report records are queryable for incident review.

Layer E. Ingestion and Document Processing Layer

Mission:
Convert manuals and manufacturer alerts into high-quality searchable chunks without blocking interactive API traffic.

Technology stack:

- Redis queue and worker process.
- PyMuPDF and unstructured for extraction.
- Embedding service and vector upsert pipeline.

Functional requirements:

1. Async ingestion for manuals and alerts.
2. Deduplication and idempotency protection.
3. Chunking strategy with overlap and metadata tagging.
4. Retry with backoff and dead-letter handling.
5. Job status tracking and operational visibility.

Required ingestion functions:

1. enqueue_ingestion_job(job_payload)
2. process_ingestion_job(job_id)
3. extract_text_from_manual(file_uri)
4. normalize_alert_payload(alert_payload)
5. chunk_document(text, chunk_size, overlap)
6. enrich_chunk_metadata(chunk, source_metadata)
7. upsert_chunks_to_vector_store(chunks)
8. publish_job_status(job_id, status)
9. move_to_dead_letter(job_id, error)

Acceptance criteria:

1. Duplicate alert payloads do not produce duplicate indexed entries.
2. Failed jobs are observable and recoverable.
3. Manual chunks include sufficient metadata for traceable citations.

Layer F. Security, Access Control, and Governance Layer

Mission:
Protect data and operations with strict identity verification, authorization boundaries, and auditable traces.

Technology stack:

- JWT token strategy using python-jose.
- Password hashing with passlib bcrypt.
- Policy middleware in FastAPI.

Functional requirements:

1. Access token and refresh token lifecycle.
2. Route-level authorization and data-level scope enforcement.
3. Retrieval pre-filtering by authorized machine scope.
4. Immutable audit events for sensitive operations.
5. Standardized 401 and 403 responses with safe error semantics.

Required security functions:

1. create_access_token(user_claims)
2. create_refresh_token(user_claims)
3. verify_token(token)
4. revoke_refresh_token(token_id)
5. get_current_user(request)
6. authorize_machine_action(user, machine_type)
7. build_scope_filter(user)
8. write_security_audit(event_type, actor, context)

Acceptance criteria:

1. Unauthorized access attempts are denied consistently across all flows.
2. Token replay and expired token behavior are tested.
3. Retrieval path applies scope filter before similarity ranking.

Layer G. Observability, Reliability, and Operations Layer

Mission:
Enable stable operation with measurable service quality, fast troubleshooting, and controlled releases.

Technology stack:

- Docker Compose services for local and staging parity.
- Prometheus instrumentation for API metrics.
- Structlog JSON logs for searchable diagnostics.

Functional requirements:

1. Health and readiness endpoints.
2. Metrics for latency, error rates, ingestion throughput, and retrieval failures.
3. Traceability for AI and retrieval decisions.
4. Alert thresholds and runbook procedures.

Required ops functions:

1. expose_health_status()
2. expose_readiness_status()
3. record_request_metrics(path, status, latency)
4. record_retrieval_metrics(top_k, latency, result_count)
5. record_ai_metrics(model, token_usage, latency)
6. record_ingestion_metrics(job_type, duration, status)
7. trigger_operational_alert(alert_type, severity, payload)

Acceptance criteria:

1. p95 chat latency and ingestion success rate are continuously visible.
2. Production incidents can be root-caused via logs and metrics without code changes.

4. Canonical Domain Objects

Report

Required fields:

1. id
2. user_id
3. machine_type
4. problem
5. cause
6. solution
7. clean_problem
8. clean_cause
9. clean_solution
10. combined_clean_text
11. source
12. created_at
13. embedding_ref

Knowledge Chunk

Required fields:

1. id
2. chunk_text
3. embedding
4. source
5. machine_type
6. timestamp
7. section (optional)
8. page (optional)
9. validation_status (optional)

Chat Audit Trace

Required fields:

1. trace_id
2. user_id

3. query_text_hash
4. retrieved_chunk_ids
5. model_name
6. response_id
7. confidence_score
8. created_at

5. Event Flow Examples

Event Flow 1: Report Reformulation and Approval

Trigger:
Repairer starts report submission in chat.

Flow:

1. Frontend submits structured report fields to reformulation endpoint.
2. Backend verifies token and machine authorization.
3. AI service reformulates fields and returns cleaned content.
4. Frontend shows cleaned content for review.
5. User iterates with Modify instructions if needed.
6. On Approve, backend computes combined text and embedding.
7. Backend stores report in PostgreSQL and vector record in Chroma.
8. Success response returns report ID and citation-ready metadata.

Edge events:

1. Unauthorized machine returns 403.
2. AI provider timeout returns retriable error and preserves draft state.

Event Flow 2: Solution Querying for a Problem

Trigger:
Repairer asks technical troubleshooting question in chat.

Flow:

1. Frontend posts query to chat endpoint.
2. Backend extracts user scope and builds metadata filter.
3. Query embedding is generated.
4. Vector search retrieves authorized top-k chunks.
5. Confidence score is computed from retrieval distribution.
6. AI generates grounded answer using retrieved context.
7. Backend maps citation IDs and persists audit trace.
8. Frontend displays diagnosis, causes, solutions, and citations.

Edge events:

1. Low confidence triggers clarification questions.
2. No retrieval results triggers explicit insufficient-context response.

Event Flow 3: Manufacturer Alert Ingestion

Trigger:
Manufacturer sends webhook alert payload.

Flow:

1. Backend validates webhook authentication and idempotency key.
2. New events are queued as async ingestion jobs.
3. Worker normalizes alert payload and extracts structured fields.
4. Embeddings are generated and chunks stored in vector DB.
5. Metadata and processing status saved in relational DB.
6. Job final status set to completed and observable in admin endpoint.

Edge events:

1. Duplicate idempotency key returns already-processed response.
2. Parsing failure sends job to dead-letter with error details.

Event Flow 4: Manual Document Processing for New Machine

Trigger:
Admin uploads new machine manual.

Flow:

1. Backend records machine metadata and enqueues ingestion.
2. Worker extracts text and splits into chunks by target token window.
3. Each chunk is tagged with machine_type, source, section, and page.
4. Embeddings generated and upserted to vector DB.
5. Ingestion summary persisted and status exposed for monitoring.

Edge events:

1. Unsupported file format returns validation error.
2. Partial chunk failures are retried with bounded attempts.

Event Flow 5: Unauthorized Query Attempt

Trigger:
User requests help for machine outside assigned permissions.

Flow:

1. Backend authorization middleware detects scope violation.
2. Request fails with 403 and standardized error object.
3. Security audit event is written with actor and machine context.
4. Frontend shows clear non-technical rejection message.

6. Release Plan by Capability

Phase 1: Core Security and Contract Completion

Deliver:

1. Auth routes and token lifecycle.
2. Report reformulation and commit endpoints.
3. Scope-aware metadata filtering in retrieval path.

Phase 2: Grounded Response Quality

Deliver:

1. Citation mapping and confidence scoring.
2. Clarification fallback for low-confidence responses.
3. Chat audit trace persistence.

Phase 3: Async Ingestion Reliability

Deliver:

1. Redis worker and ingestion orchestration.
2. Idempotency store and dead-letter handling.
3. Admin job status endpoint.

Phase 4: Operational Hardening

Deliver:

1. Metrics and alerts for API, AI, and ingestion paths.
2. Safety guardrails for high-risk instructions.
3. Evaluation harness and quality gates in CI.

7. Test Matrix

Functional tests:

1. Report reformulation success and approval loop.
2. Report modification iteration behavior.
3. Chat retrieval with citations.
4. Manufacturer alert ingestion success path.
5. Manual parsing and vector indexing path.

Security tests:

1. Invalid token rejected on protected endpoints.
2. Unauthorized machine action rejected.
3. Retrieval cannot return out-of-scope chunks.
4. Duplicate ingestion requests handled idempotently.

Reliability tests:

1. AI timeout retry behavior.
2. Queue retry and dead-letter routing.
3. Recovery after worker restart.

8. Implementation Checklist

1. Implement missing routers in backend startup.
2. Replace AI stubs with provider-backed service methods.
3. Introduce canonical DTOs for report, chat, and ingestion flows.
4. Add permission middleware and retrieval scope filter.
5. Build ingestion worker process and job status model.
6. Add audit trace persistence for chat and security events.
7. Integrate frontend with real backend APIs.
8. Add observability hooks and alert thresholds.

9. Definition of Done

1. Every layer requirements in this document are mapped to implemented functions.
2. End-to-end tests pass for report, query, and ingestion core flows.
3. Security tests pass for unauthorized access and token failures.
4. Metrics and logs are available in staging.
5. Product and technical stakeholders validate event-flow behavior in demo.